



COMPREHENSIVE GUIDE TO MODERN ANGULAR CHANGE DETECTION



Author ANaik



Last Updated 2026-01-02



Read Time 25 minutes

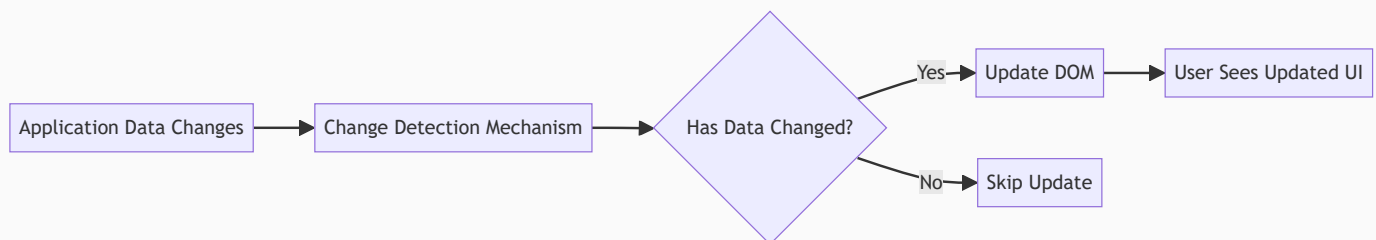
1. Understanding Change Detection: The Foundation

1.1 What is Change Detection?

The Restaurant Analogy: Imagine a restaurant where the menu board displays current specials. Change detection is like the waiter who checks if the kitchen has updated the specials and then updates the menu board for customers to see.

In Angular terms:

- **Kitchen** = Your application data (TypeScript variables)
- **Menu Board** = Your UI (HTML template)
- **Waiter** = Change Detection mechanism
- **Updating the Board** = Re-rendering the DOM



1.2 Why Do We Need It?

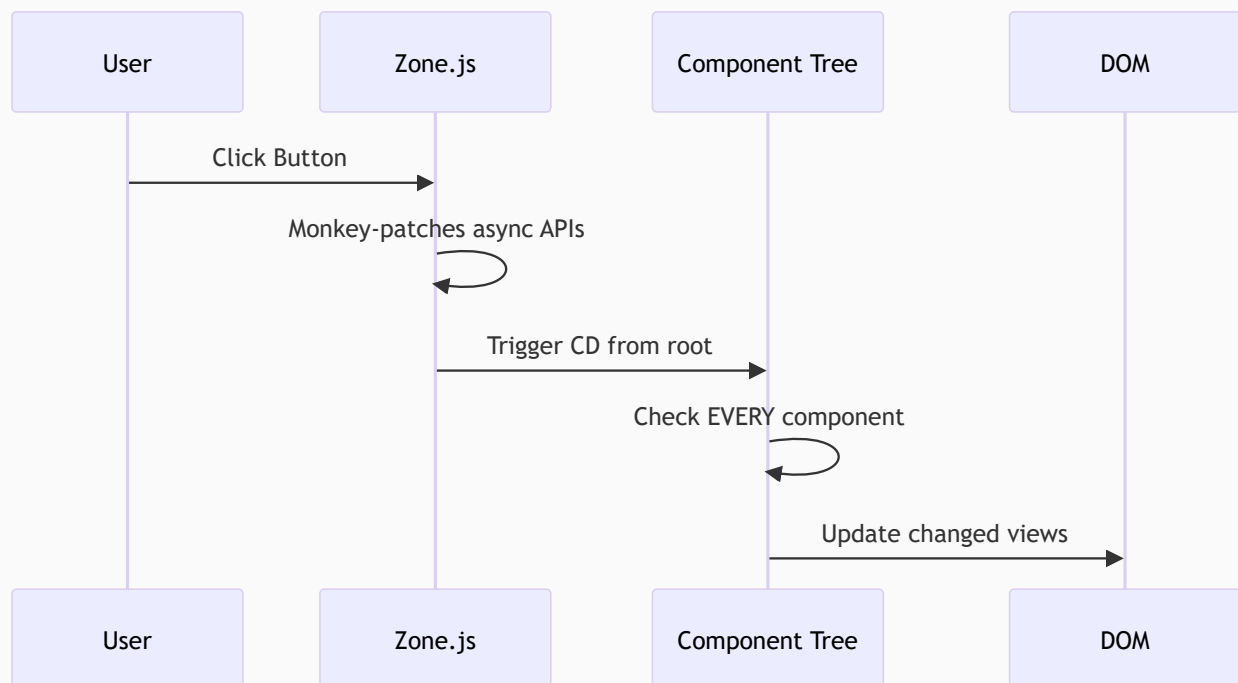
Angular applications are **reactive** - the UI needs to reflect the current state of your data. When a button is clicked, an API response arrives, or a timer fires, the UI must update. Change detection is the bridge between your data model and what users see.

2. Legacy vs Modern: The Paradigm Shift

2.1 Zone.js-Based Change Detection (Legacy Approach)

The Security Guard Analogy: Zone.js is like a security guard who monitors EVERY door in a building. Whenever anyone enters or leaves through ANY door, the guard checks ALL rooms to see if anything changed.

How Zone.js Works



Key Characteristics:

- Monkey-patches all async APIs (setTimeout, HTTP, events)
- Triggers change detection from the **root** component
- Checks **every** component in the tree (unless OnPush is used)
- **Implicit** - happens automatically, but unpredictably
- Can be inefficient with large component trees

```
// Legacy Zone-based approach  
export class LegacyComponent {
```

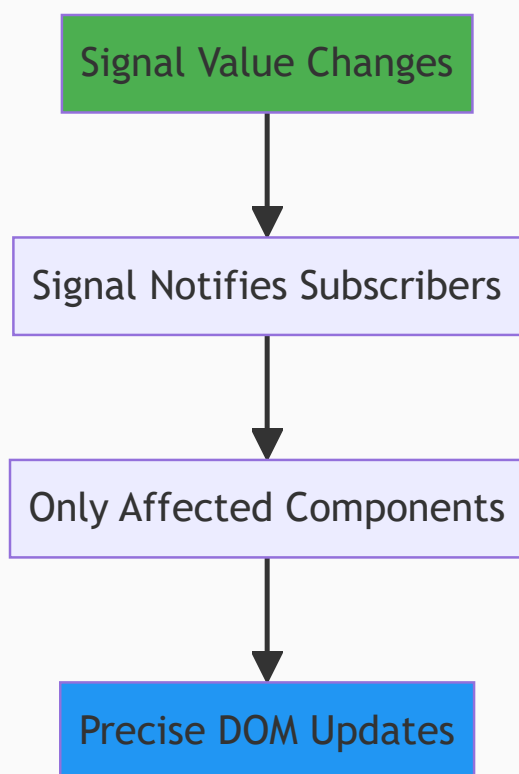
```
counter = 0;

increment() {
  this.counter++; // Zone.js detects this and triggers CD
}
```

2.2 Signal-Based Change Detection (Modern Approach)

The Smart Home Analogy: Signals are like smart home sensors. Each light bulb knows exactly which motion sensor controls it. When a specific sensor detects movement, **ONLY** that light bulb turns on - not every light in the house.

How Signals Work



Key Characteristics:

- **Explicit** reactive primitives (`signal()` , `computed()` , `effect()`)
- **Granular** change detection - only affected components update

- **Predictable** - you know exactly when and why CD runs
- **Efficient** - no need to check entire component tree
- **Optional** - can coexist with Zone.js

```
// Modern Signal-based approach
import { Component, signal, computed } from '@angular/core';

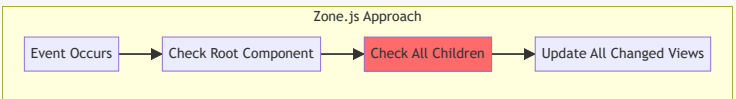
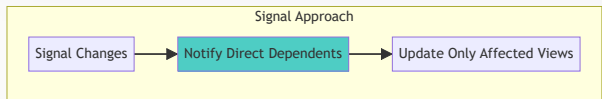
export class ModernComponent {
  counter = signal(0);
  doubleCounter = computed(() => this.counter() * 2);

  increment() {
    this.counter.update(val => val + 1);
    // Only components using this signal update
  }
}
```

2.3 Side-by-Side Comparison

Aspect	Zone.js (Legacy)	Signals (Modern)
Detection Strategy	Top-down tree traversal	Reactive graph updates
Granularity	Component-level	Signal-level
Predictability	Implicit (mysterious)	Explicit (clear)
Performance	Can be wasteful	Highly optimized

Aspect	Zone.js (Legacy)	Signals (Modern)
Learning Curve	Easier initially	Requires understanding reactivity
Bundle Size	Larger (Zone.js ~14KB)	Smaller (signals ~3KB)



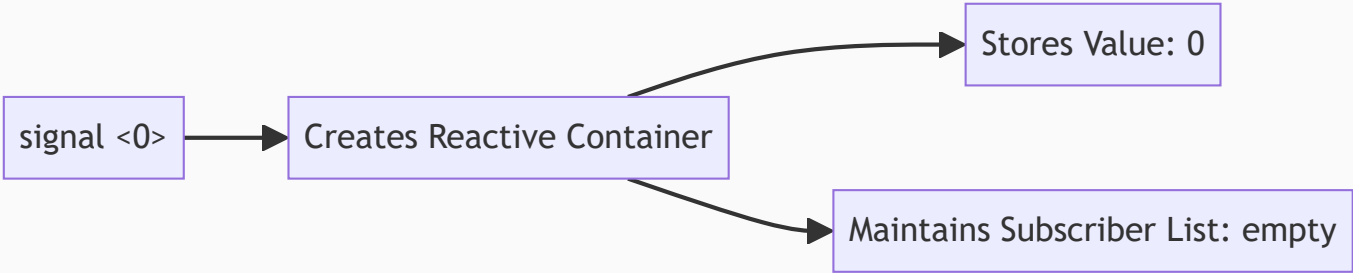
3. Under the Hood: How Modern Change Detection Works

3.1 Step-by-Step: Signal Reactivity Mechanism

The Newsletter Subscription Analogy: Signals work like email newsletters. You subscribe to specific topics (signals), and you only get emails when THOSE topics have updates - not every time anyone posts anything anywhere.

Step 1: Signal Creation

```
const count = signal(0);
```

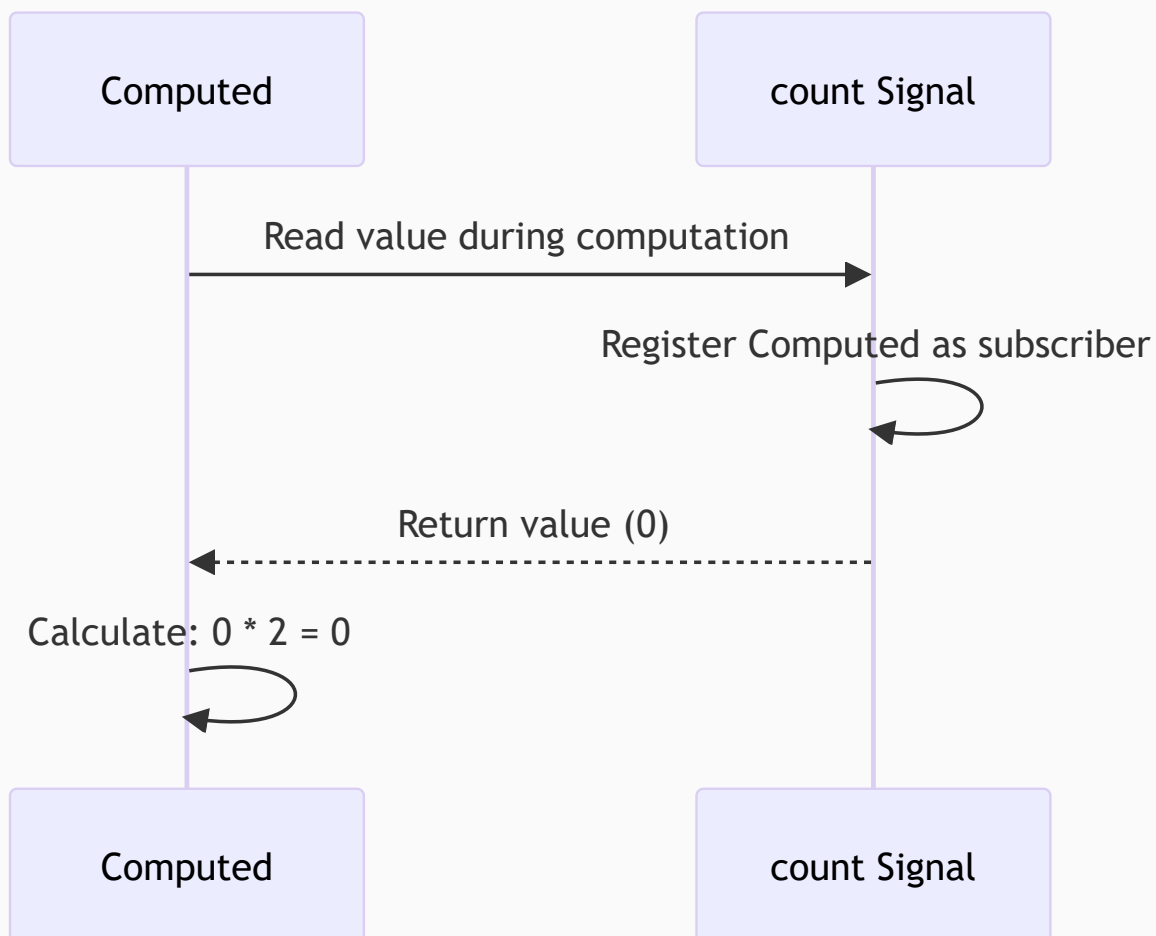


What happens internally:

1. Angular creates a reactive wrapper around the value
2. Initializes an empty list of subscribers (effects/computed/components)
3. Assigns a unique ID to track the signal

Step 2: Creating Dependencies (Computed Signals)

```
const doubleCount = computed(() => count() * 2);
```

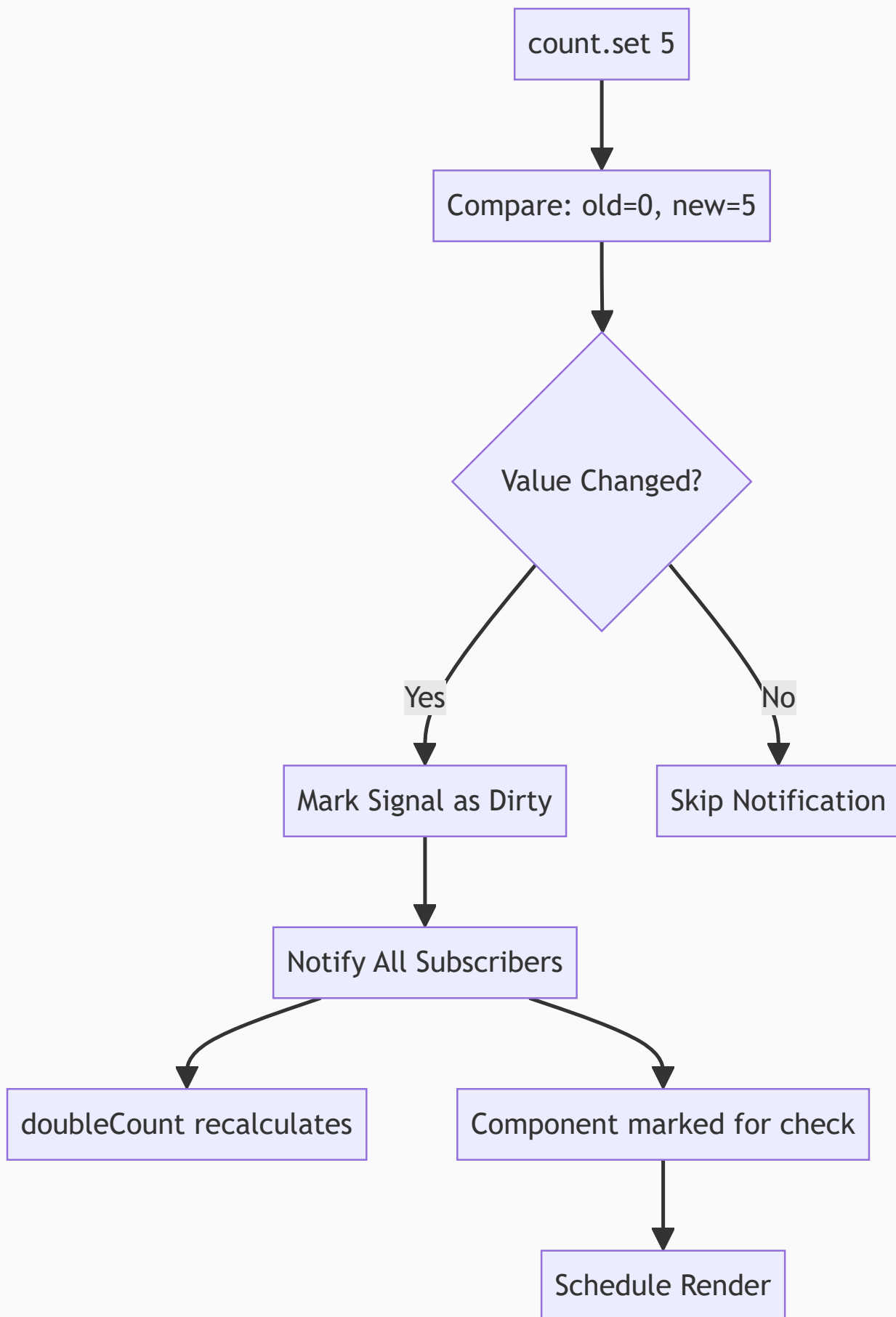


What happens internally:

1. During `computed()` execution, Angular tracks which signals are read
2. Those signals add the computed signal to their subscriber lists
3. A dependency graph is built automatically

Step 3: Signal Update

```
count.set(5);  
// or  
count.update(val => val + 1);
```

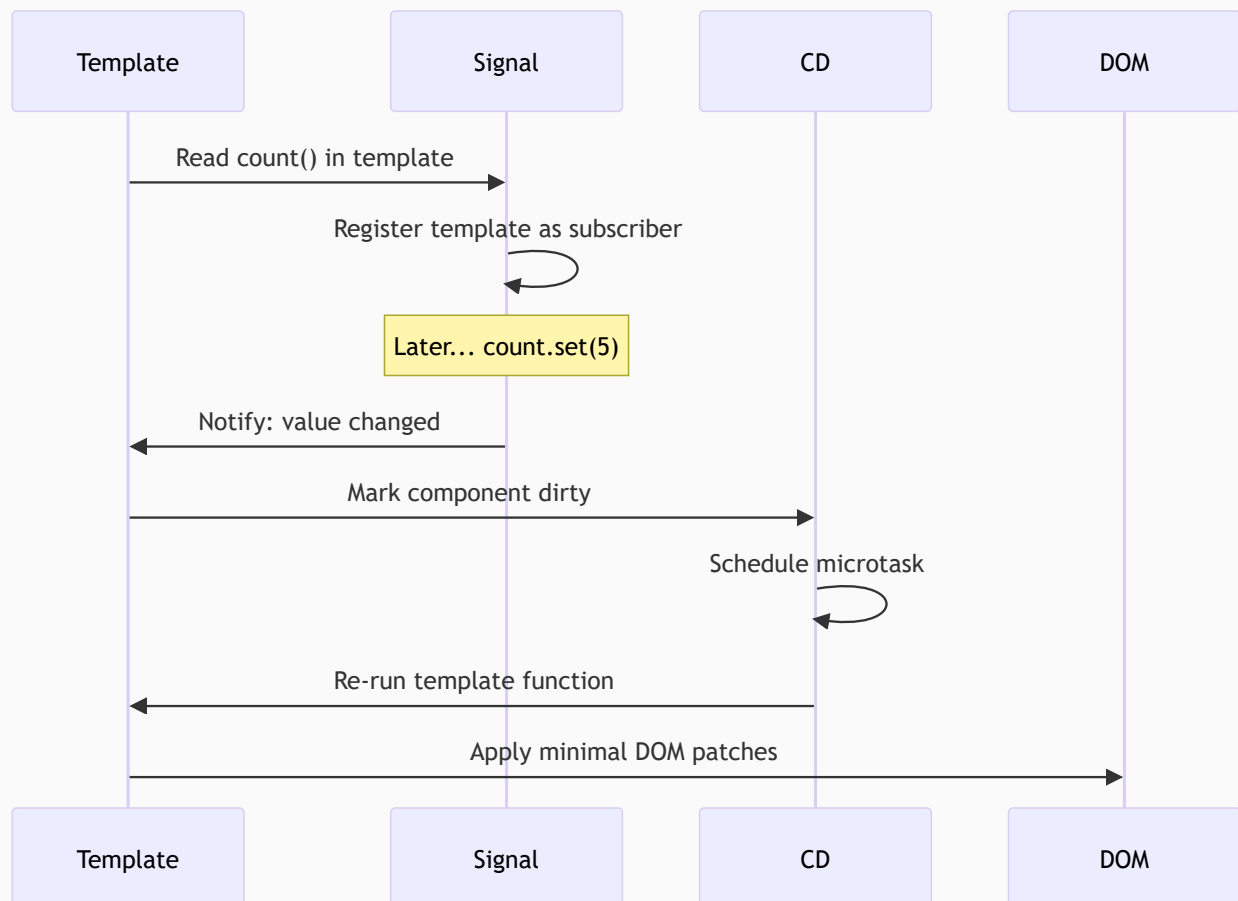


What happens internally:

1. **Comparison:** Angular checks if the new value differs from the old value
2. **Marking:** If changed, marks the signal as “dirty”
3. **Notification:** Immediately notifies all subscribers synchronously
4. **Propagation:** Computed signals recalculate; components schedule updates
5. **Batching:** Multiple updates in the same task are batched together

Step 4: Component Update

```
@Component({  
  template: `<p>Count: {{ count() }}</p>`  
})  
class MyComponent {  
  count = signal(0);  
}
```

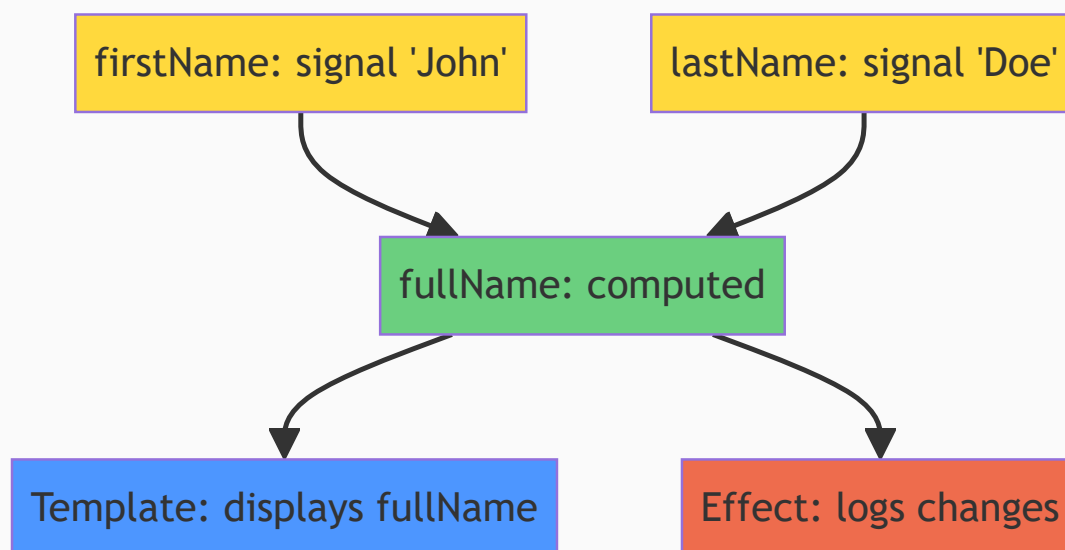


What happens internally:

1. When the template reads `count()` , it registers as a subscriber
2. When `count` changes, the template is notified
3. Angular marks the component for check
4. During the next change detection cycle, only the affected binding updates
5. **Zone-less mode**: No Zone.js needed - signals drive everything

3.2 The Reactive Graph

Signals create a **dependency graph** that Angular uses to optimize updates:



When `firstName.set('Jane')` :

1. Only `firstName` is marked dirty
2. Only `fullName` (which depends on it) recalculates
3. Only the template binding and effect are notified
4. `lastName` and its dependents are untouched

4 Performance Optimization Strategies

4.1 1. Use Computed Signals for Derived State

✗ Bad: Recalculating on Every Check**

```
export class BadComponent {
  items = signal([1, 2, 3, 4, 5]);

  // This function runs EVERY time the template is checked
  get expensiveCalculation() {
    console.log('Calculating...');
    return this.items().reduce((sum, item) => sum + item, 0);
  }
}
```

✓ Good: Memoized with Computed

```
export class GoodComponent {
  items = signal([1, 2, 3, 4, 5]);

  // Only recalculates when items actually changes
  total = computed(() => {
    console.log('Calculating...');
    return this.items().reduce((sum, item) => sum + item, 0);
  });
}
```

4.2 2. Avoid Unnecessary Signal Reads

✗ Bad: Reading Signals Outside Reactive Context

```
export class BadComponent {
  count = signal(0);
```

```
logCount() {  
  // Reading the signal, but not in a reactive context  
  // Creates unnecessary dependency  
  console.log(this.count());  
}  
}
```

✓ Good: Use Untracked or Direct Access

```
import { untracked } from '@angular/core';  
  
export class GoodComponent {  
  count = signal(0);  
  
  logCount() {  
    // Read without creating dependency  
    console.log(untracked(() => this.count()));  
  }  
}
```

4.3 3. Batch Updates with update()

✗ Bad: Multiple Set Calls

```
// Triggers 3 separate change detection cycles  
firstName.set('John');  
lastName.set('Doe');  
age.set(30);
```

✓ Good: Batch Operations**

```
// Use update for atomic changes
userState.update(state => ({
  ...state,
  firstName: 'John',
  lastName: 'Doe',
  age: 30
}));
```

4 OnPush Strategy with Signals

```
@Component({
  selector: 'app-optimized',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div>Count: {{ count() }}</div>
    <div>Double: {{ doubled() }}</div>
  `
})
export class OptimizedComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  // With OnPush + Signals, you get:
  // 1. No unnecessary checks from parent
  // 2. Precise updates from signal changes
}
```

4.6 Performance Impact Visualization

With Signals + OnPush

User Action



Check 2 Components



Update 2 Components

Without Optimization

User Action



Check 100 Components



Update 2 Components

5. Anti-Patterns: What NOT to Do

5.1 ❌ Anti-Pattern 1: Converting Everything to Signals

The Problem: Signal-ifying every single variable creates unnecessary complexity.

```
// OVERKILL - Don't do this!
export class OverEngineeredComponent {
  // These don't need to be signals
  readonly title = signal('My Component');
  readonly staticConfig = signal({ color: 'blue', size: 'large' });
  private readonly componentId = signal('abc-123');

  // This constant changes based on signal - unnecessary
  readonly PI = signal(3.14159);
}
```

Why It's Bad:

- Adds boilerplate for no benefit
- Static/constant values don't need reactivity
- Hurts readability and maintainability

✓ Better Approach:

```
export class SensibleComponent {
  // Simple values don't need signals
  readonly title = 'My Component';
  readonly staticConfig = { color: 'blue', size: 'large' };
  private readonly componentId = 'abc-123';

  // Only reactive data should be signals
  userCount = signal(0);
  isLoading = signal(false);
}
```

```

// Don't blindly convert all inputs!
@Component({
  selector: 'app-user-card'
})
export class UserCardComponent {
  // ❌ Unnecessary - simple input is fine
  userId = input.required<string>();
  userName = input.required<string>();
  userEmail = input.required<string>();
  userPhone = input<string>();
  userAddress = input<string>();
}

```

✅ Use Regular Inputs for Simple Pass-Through:

```

@Component({
  selector: 'app-user-card'
})
export class UserCardComponent {
  // Regular inputs for simple display
  @Input({ required: true }) userId!: string;
  @Input({ required: true }) userName!: string;

  // Signal inputs when you need to derive/compute
  userEmail = input.required<string>();
  emailDomain = computed(() => this.userEmail().split('@')[1]);
}

```

5.3 ❌ Anti-Pattern 3: Abandoning RxJS Entirely

The Myth: “Signals replace RxJS, so I should remove all observables.”

The Reality: Signals and RxJS serve different purposes and work great together!

```
// ❌ Bad: Fighting RxJS with signals
export class BadComponent {
  searchQuery = signal('');
  results = signal<any[]>([]);

  constructor(private http: HttpClient) {
    // Trying to manually handle async with signals - messy!
    effect(() => {
      const query = this.searchQuery();
      this.http.get(`/api/search?q=${query}`).subscribe(data => {
        this.results.set(data);
      });
      // No debouncing, no cancellation, memory leaks!
    });
  }
}
```

✅ Good: Use RxJS for Async, Signals for State:

```
import { toSignal } from '@angular/core/rxjs-interop';

export class GoodComponent {
  searchQuery = signal('');

  // RxJS excels at async operations
  private searchResults$ = toObservable(this.searchQuery).pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(query => this.http.get(`/api/search?q=${query}`))
  );
}
```

```
// Convert final result to signal
results = toSignal(this.searchResults$, { initialValue: [] });
}
```

5.4 ❌ Anti-Pattern 4: Triggering Unnecessary Change Detection

```
// ❌ Bad: Mutating signal content without .update()
export class BadComponent {
  items = signal([1, 2, 3]);

  addItem(item: number) {
    // This WON'T trigger change detection!
    this.items().push(item); // Mutation doesn't notify subscribers
  }
}
```

✅ Good: Immutable Updates:

```
export class GoodComponent {
  items = signal([1, 2, 3]);

  addItem(item: number) {
    // Creates new array, triggers change detection
    this.items.update(current => [...current, item]);
  }
}
```

5.5 ❌ Anti-Pattern 5: Over-Using Effects

```

// ❌ Bad: Effects for everything
export class EffectHeavyComponent {
  count = signal(0);

  constructor() {
    // Effect for simple logging - overkill
    effect(() => console.log(this.count()));

    // Effect for derived state - use computed!
    effect(() => {
      const doubled = this.count() * 2;
      // Now what? Can't return it...
    });

    // Effect that updates another signal - infinite loop risk!
    effect(() => {
      this.count.set(this.count() + 1); // ⚡ Infinite loop!
    });
  }
}

```

✅ Good: Use Effects Sparingly:

```

export class GoodComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2); // ✅ Derived state

  constructor() {
    // ✅ Effect only for side effects (logging, analytics, etc.)
    effect(() => {
      console.log('Count changed:', this.count());
      this.analytics.track('count_changed', this.count());
    });
  }
}

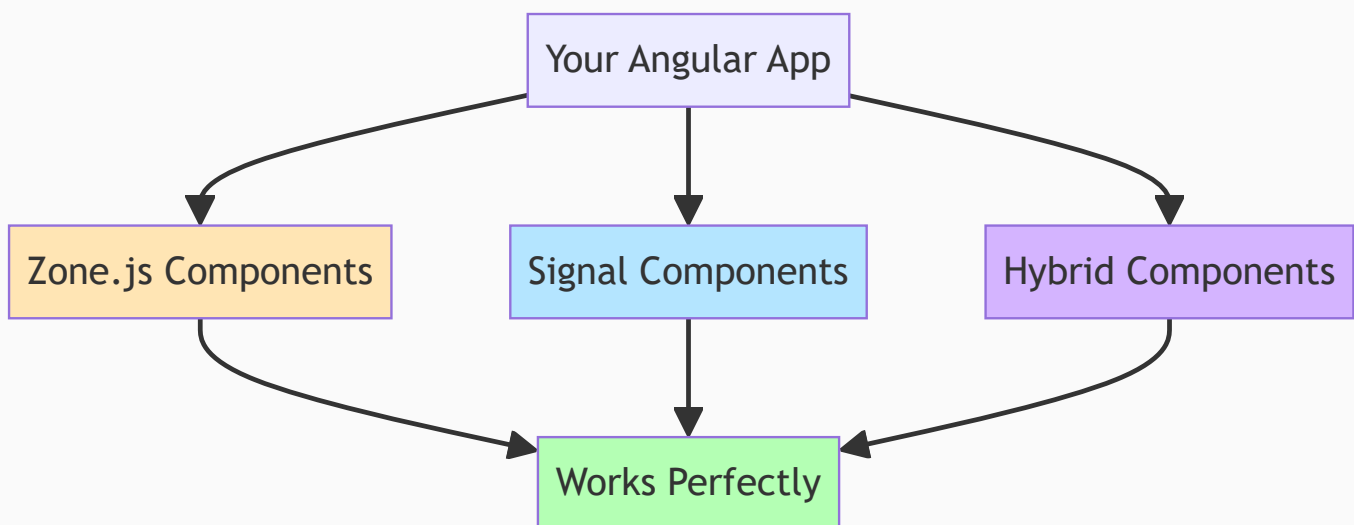
```

```
}  
}
```

6. Backward Compatibility & Migration Philosophy

6.1 Angular's Compatibility Promise

The Renovation Analogy: Angular's migration strategy is like renovating a house while people live in it. You don't force everyone to move out; you upgrade room by room, and the old and new rooms work together perfectly.



6.2 How Backward Compatibility Works

1. Signal Components Work with Zone.js

```
// Parent: Old Zone.js component  
@Component({  
  selector: 'app-parent',  
  template: `  
    <app-modern-child [count]="zoneCount"></app-modern-child>  
  `,  
})
```

```

export class ParentComponent {
  zoneCount = 5; // Regular property

  increment() {
    this.zoneCount++; // Zone.js detects this
  }
}

// Child: New Signal component
@Component({
  selector: 'app-modern-child',
  template: `

Count: {{ count() }}

`
})
export class ModernChildComponent {
  count = input<number>(0); // Signal input

  // Angular automatically bridges the gap!
}

```

What Angular Does Behind the Scenes:

1. Detects input change via Zone.js
2. Automatically updates the signal input
3. Triggers signal-based change detection in child
4. Both paradigms coexist harmoniously

2. Gradual Migration Support

```

// You can mix and match in the same component!
export class HybridComponent {
  // Legacy: Regular property
  oldCounter = 0;

  // Modern: Signal

```

```

newCounter = signal(0);

// Legacy: Input decorator
@Input() legacyInput!: string;

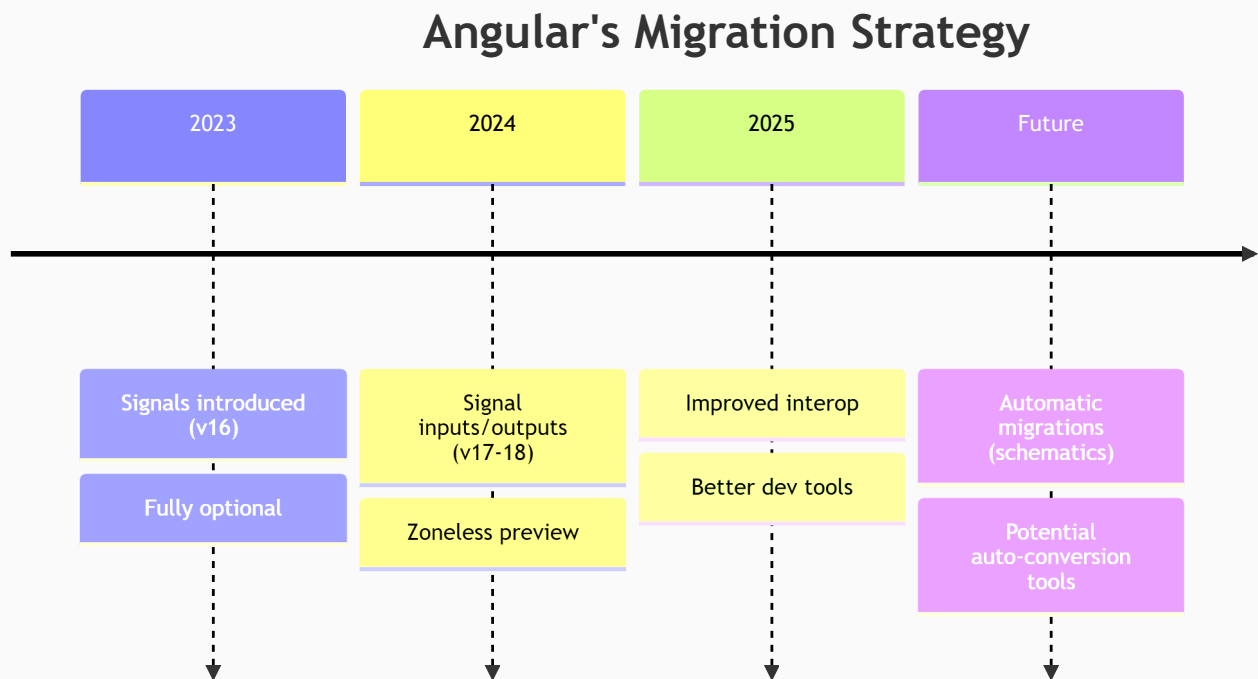
// Modern: Signal input
modernInput = input<string>('');

// Both work perfectly together!
}

```

6.3 Google's Migration Philosophy

Key Principle: “You don’t migrate; Angular migrates FOR you.”



What Google Has Committed To:

1. **No Breaking Changes:** Old code continues to work
2. **Automatic Interop:** Signals and Zone.js communicate seamlessly

3. **Migration Tools:** Schematics to help (when ready)
4. **Opt-In Future:** You choose when to adopt signals

6.4 The Auto-Migration Future

Hypothetical Scenario (based on Angular team's direction):

```
# Future Angular CLI command (hypothetical)
ng update @angular/core --migrate-to-signals

# This MIGHT automatically convert:
# - @Input() → input()
# - @Output() → output()
# - Class properties → signals (where beneficial)
```

Why You Shouldn't Wait for Auto-Migration:

- Understanding signals makes you a better Angular developer
- You can optimize as you go
- Auto-migration won't always make the best architectural decisions

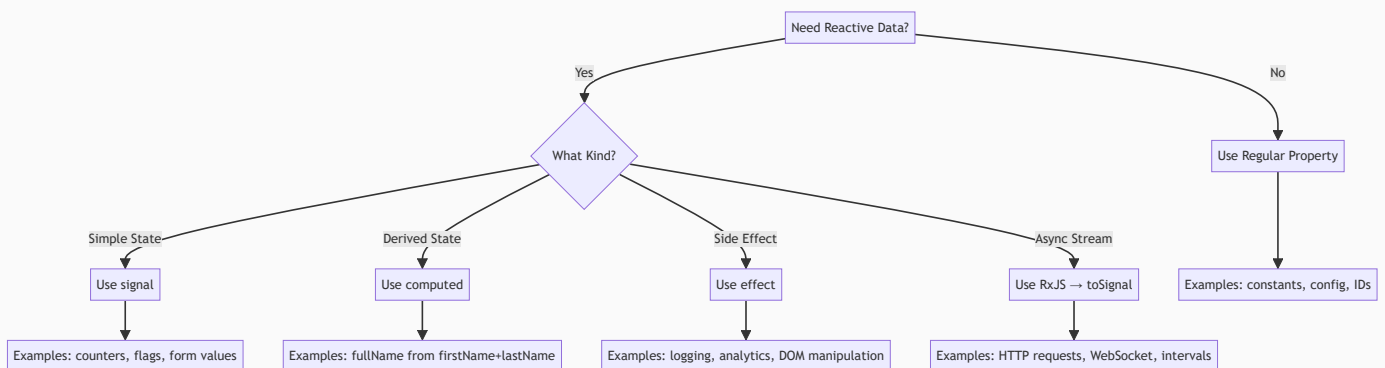
6.5 Guaranteed Compatibility Scenarios

Scenario	Compati lity	Notes
Signal child, Zone parent	 Perfect	Angular bridges automatically
Zone child, Signal parent	 Perfect	Works seamlessly

Scenario	Compati bility	Notes
Mixed inputs in one component	✓ Perfect	No conflicts
RxJS + Signals	✓ Perfect	Use <code>toSignal</code> / <code>toObservable</code>
OnPush + Signals	✓ Perfect	Recommended combination
Legacy services in Signal components	✓ Perfect	Inject and use normally

7. When to Use Signals (and When Not To)

7.1 Decision Tree



7 ✓ Use Signals When

1. Component State That Changes

```
export class TodoComponent {  
  // ✅ Perfect for signals  
  todos = signal<Todo[]>([]);  
  filter = signal<'all' | 'active' | 'completed'>('all');  
  isLoading = signal(false);  
}
```

2. Derived/Computed Values

```
export class ShoppingCartComponent {  
  items = signal<CartItem[]>([]);  
  
  // ✅ Computed automatically updates  
  total = computed(() =>  
    this.items().reduce((sum, item) => sum + item.price * item.quantity, 0)  
  );  
  
  itemCount = computed(() => this.items().length);  
  
  isEmpty = computed(() => this.itemCount() === 0);  
}
```



3. Inputs That Drive Computations

```
export class UserProfileComponent {  
  // ✅ Signal input when you need to derive from it  
  userId = input.required<string>();  
}
```

```

// Computed based on input
profileUrl = computed(() => `/api/users/${this.userId()}/profile`);

// Use in effects
constructor(private http: HttpClient) {
  effect(() => {
    this.loadProfile(this.userId());
  });
}
}

```

4. Form State (Sometimes)

```

export class SearchComponent {
  // ✅ Good for simple form state
  searchQuery = signal('');
  selectedCategory = signal('all');

  // Computed filter
  filteredResults = computed(() =>
    this.applyFilter(this.searchQuery(), this.selectedCategory())
  );
}

```

7 ❌ Don't Use Signals When

1. Static/Constant Values

```

export class ConfigComponent {
  // ❌ Don't use signals for these
  readonly APP_NAME = signal('My App');
}

```

```

readonly VERSION = signal('1.0.0');
private readonly API_URL = signal('https://api.example.com');

// ✅ Use regular properties instead
readonly APP_NAME = 'My App';
readonly VERSION = '1.0.0';
private readonly API_URL = 'https://api.example.com';
}

```

2. Complex Async Workflows

```

// ❌ Bad: Complex async with signals
export class BadAsyncComponent {
  data = signal(null);

  loadData() {
    effect(() => {
      // Complex retry logic, error handling, cancellation?
      // This gets messy fast...
    });
  }
}

// ✅ Good: Use RxJS for complex async
data$ = this.http.get('/api/data').pipe(
  retry(3),
  catchError(err => of(null)),
  shareReplay(1)
);

data = toSignal(this.data$);
}

```

3. Event Streams

```

// ❌ Don't use signals for event streams
export class ChatComponent {
  // This is an event stream, not state
  messages = signal<Message[]>([]);

  // Trying to handle WebSocket with signals - awkward!

  // ✅ Use RxJS for streams
  private messages$ = this.websocket.messages$;
  messages = toSignal(this.messages$, { initialValue: [] });
}

```

4. Simple Display-Only Inputs

```

@Component({
  selector: 'app-label',
  template: `<span>{{ text }}</span>`
})
export class LabelComponent {
  // ❌ Overkill for simple pass-through
  text = input<string>('');

  // ✅ Regular input is simpler
  @Input() text = '';
}

```

7.6 Signals + RxJS: The Best of Both Worlds

```

export class BestPracticeComponent {
  // Signal for simple state

```

```

searchTerm = signal('');

// RxJS for async operations
private searchResults$ = toObservable(this.searchTerm).pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.api.search(term)),
  catchError(() => of([]))
);

// Back to signal for template binding
results = toSignal(this.searchResults$, { initialValue: [] });

// Computed for derived state
hasResults = computed(() => this.results().length > 0);
}

```

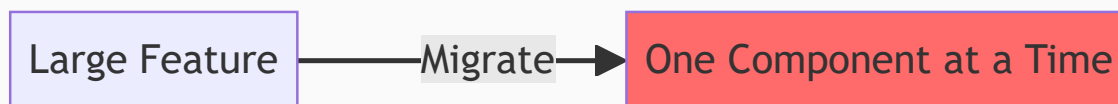
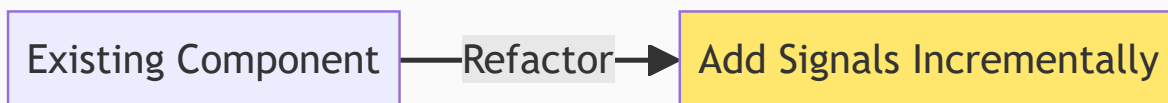
7.7 Quick Reference Matrix

Use Case	Use Signals?	Use RxJS?	Use Both?
Component state	✓ Yes	✗ No	-
Derived values	✓ computed()	✗ No	-
HTTP requests	✗ No	✓ Yes	✓ toSignal
WebSockets	✗ No	✓ Yes	✓ toSignal
Form controls	⚠ Sometimes	✓ Often	✓ Hybrid
Route params	✗ No	✓ Yes	✓ toSignal

Use Case	Use Signals?	Use RxJS?	Use Both?
Simple inputs	✗ @Input	-	-
Computed inputs	✓ input()	-	-
Constants	✗ No	✗ No	-
Side effects	✓ effect()	✗ No	-

8. Best Practices & Real-World Guidance

8.1 Principle 1: Start Small, Grow Gradually



Example Migration Path:

```

// Week 1: Start with state
export class TodoComponent {
  todos = signal<Todo[]>([]);

  @Input() userId!: string; // Keep legacy input

```

```

}

// Week 2: Add computed
export class TodoComponent {
  todos = signal<Todo[]>([]);
  completedCount = computed(() =>
    this.todos().filter(t => t.completed).length
  );

  @Input() userId!: string;
}

// Week 3: Convert input when beneficial
export class TodoComponent {
  todos = signal<Todo[]>([]);
  completedCount = computed(() =>
    this.todos().filter(t => t.completed).length
  );

  userId = input.required<string>(); // Now signals-based

  userTodos = computed(() =>
    this.todos().filter(t => t.userId === this.userId())
  );
}

```

8.2 Principle 2: Signals for State, RxJS for Streams

The File Cabinet Analogy:

- **Signals** = Filing cabinet drawers (current state)
- **RxJS** = Conveyor belt (stream of events)

```

export class NotificationComponent {
  // ✅ Signal: Current state of notifications
  notifications = signal<Notification[]>([]);
  unreadCount = computed(() =>
    this.notifications().filter(n => !n.read).length
  );

  // ✅ RxJS: Stream of incoming notifications
  private newNotifications$ = this.websocket.notifications$.pipe(
    tap(notification => {
      // Update signal when new notification arrives
      this.notifications.update(current => [...current, notification]);
    })
  );

  constructor() {
    // Subscribe to the stream
    this.newNotifications$.subscribe();
  }
}

```

8.3 Principle 3: Computed Over Effects

```

// ❌ Anti-pattern: Using effect for derived state
export class BadComponent {
  firstName = signal('John');
  lastName = signal('Doe');
  fullName = signal('');

  constructor() {
    effect(() => {
      // ❌ Don't do this - updating signals in effects is fragile
      this.fullName.set(`${this.firstName()} ${this.lastName()}`);
    });
  }
}

```

```

    });
  }
}

// ✅ Best practice: Use computed
export class GoodComponent {
  firstName = signal('John');
  lastName = signal('Doe');

  // ✅ Automatically updates, no effects needed
  fullName = computed(() => `${this.firstName()} ${this.lastName()}`);
}

```

8.4 Principle 4: Immutability with Collections

```

export class TodoListComponent {
  todos = signal<Todo[]>([]);

  // ❌ Bad: Mutation doesn't trigger updates
  addTodoBad(todo: Todo) {
    this.todos().push(todo); // No change detection!
  }

  // ✅ Good: Immutable update
  addTodo(todo: Todo) {
    this.todos.update(current => [...current, todo]);
  }

  // ✅ Good: Update specific item
  toggleTodo(id: string) {
    this.todos.update(current =>
      current.map(todo =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      )
    );
  }
}

```

```

    );
}

// ✅ Good: Remove item
removeTodo(id: string) {
    this.todos.update(current => current.filter(todo => todo.id !== id));
}

```

8.5 Principle 5: Effect Cleanup

```

export class AutoSaveComponent {
    formData = signal({ name: '', email: '' });

    constructor(private api: ApiService) {
        // ✅ Effects automatically clean up, but be careful with subscription
        effect((onCleanup) => {
            const data = this.formData();

            // Debounced save
            const timeoutId = setTimeout(() => {
                this.api.save(data).subscribe();
            }, 1000);

            // Cleanup previous timeout
            onCleanup(() => clearTimeout(timeoutId));
        });
    }
}

```

8.6 Principle 6: Smart Component Pattern with Signals

```

// Smart Component (Container)
@Component({
  selector: 'app-user-dashboard',
  template: `
    <app-user-profile
      [user]="currentUser()"
      (userUpdated)="updateUser($event)" />

    <app-user-stats
      [stats]="userStats()" />
  `
})
export class UserDashboardComponent {
  // Signal state managed here
  private userId = signal('123');

  // Async data fetching with RxJS
  private user$ = toObservable(this.userId).pipe(
    switchMap(id => this.api.getUser(id))
  );

  currentUser = toSignal(this.user$);

  // Computed derived state
  userStats = computed(() => {
    const user = this.currentUser();
    return user ? this.calculateStats(user) : null;
  });

  updateUser(updatedUser: User) {
    // Update signal, triggers recomputation
    this.currentUser.set(updatedUser);
  }
}

```

```
// Presentational Component (Dumb)
@Component({
  selector: 'app-user-profile',
  template: `...`
})
export class UserProfileComponent {
  // Simple inputs for presentation
  @Input() user?: User;
  @Output() userUpdated = new EventEmitter<User>();

  // No signals needed here - just displays data
}
```

8.7 Principle 7: Type Safety with Signals

```
//  Define strong types
interface UserState {
  id: string;
  name: string;
  email: string;
  isActive: boolean;
}

export class UserComponent {
  // Type-safe signal
  user = signal<UserState>({
    id: '',
    name: '',
    email: '',
    isActive: false
  });

  // Type-safe computed
  userDisplayName = computed(()> string => {
```

```

    const u = this.user();
    return u.isActive ? u.name : `${u.name} (Inactive)`;
  });

  // Type-safe updates
  activateUser() {
    this.user.update((current): UserState => ({
      ...current,
      isActive: true
    }));
  }
}

```

8.8 Principle 8: Avoid Signal Overuse in Templates

```

@Component({
  template: `
    <!-- ❌ Bad: Reading signal multiple times -->
    <div *ngIf="user()">
      <h1>{{ user().name }}</h1>
      <p>{{ user().email }}</p>
      <span>{{ user().role }}</span>
    </div>

    <!-- ✅ Good: Read once, use alias -->
    <div *ngIf="user() as currentUser">
      <h1>{{ currentUser.name }}</h1>
      <p>{{ currentUser.email }}</p>
      <span>{{ currentUser.role }}</span>
    </div>

    <!-- ✅ Even better: Use computed if accessed frequently -->
    ,
  })

```

```
export class UserComponent {  
  user = signal<User | null>(null);  
}
```

8.9 Real-World Example: Complete Feature

```
//  Complete example following all best practices  
@Component({  
  selector: 'app-product-catalog',  
  changeDetection: ChangeDetectionStrategy.OnPush,  
  template: `  
    <div class="catalog">  
      <input  
        [ngModel]="searchQuery()"  
        (ngModelChange)="searchQuery.set($event)"  
        placeholder="Search products..." />  
  
      <select  
        [ngModel]="selectedCategory()"  
        (ngModelChange)="selectedCategory.set($event)">  
        <option value="all">All Categories</option>  
        <option value="electronics">Electronics</option>  
        <option value="clothing">Clothing</option>  
      </select>  
  
      <div class="results">  
        <p>Found {{ filteredProducts().length }} products</p>  
  
        <div *ngFor="let product of filteredProducts()" class="product-card">  
          <h3>{{ product.name }}</h3>  
          <p>{{ product.price | currency }}</p>  
        </div>  
      </div>  
    </div>
```

```

        <div *ngIf="isLoading()" class="loading">Loading...</div>
        <div *ngIf="error()" class="error">{{ error() }}</div>
    </div>
    ,
  })
}

export class ProductCatalogComponent {
  private api = inject(ProductApiService);

  // 1. Simple state as signals
  searchQuery = signal('');
  selectedCategory = signal<string>('all');
  isLoading = signal(false);
  error = signal<string | null>(null);

  // 2. Async data with RxJS → Signal
  private products$ = this.api.getProducts().pipe(
    tap(() => this.isLoading.set(true)),
    catchError(err => {
      this.error.set(err.message);
      return of([]);
    }),
    finalize(() => this.isLoading.set(false))
  );

  products = toSignal(this.products$, { initialValue: [] });

  // 3. Computed derived state
  filteredProducts = computed(() => {
    const query = this.searchQuery().toLowerCase();
    const category = this.selectedCategory();
    const allProducts = this.products();

    return allProducts.filter(product => {
      const matchesSearch = product.name.toLowerCase().includes(query);
      const matchesCategory = category === 'all' || product.category === category;
      return matchesSearch && matchesCategory;
    });
  });
}

```

```

});

// 4. More computed values
totalValue = computed(() =>
  this.filteredProducts().reduce((sum, p) => sum + p.price, 0)
);

hasResults = computed(() => this.filteredProducts().length > 0);

// 5. Effect only for side effects
constructor() {
  effect(() => {
    const query = this.searchQuery();
    const category = this.selectedCategory();

    // Analytics tracking
    console.log('User searched:', { query, category });

    // Could send to analytics service
    // this.analytics.track('search', { query, category });
  });
}

```

8.10 Performance Monitoring

```

export class PerformanceMonitoringComponent {
  items = signal<Item[]>([]);

  //  Monitor computation cost
  expensiveComputation = computed(() => {
    const start = performance.now();
    const result = this.items().map(item => /* expensive operation */);
    const duration = performance.now() - start;

```

```

    if (duration > 16) { // More than one frame
      console.warn('Slow computation:', duration);
    }

    return result;
  });

  //  Use untracked() for reads that shouldn't trigger recomputation
  logItems() {
    untracked(() => {
      console.log('Current items:', this.items());
    });
  }
}

```

8.11 Testing Signals

```

describe('ProductCatalogComponent', () => {
  it('should filter products by search query', () => {
    const component = new ProductCatalogComponent();

    // Set up initial data
    component.products.set([
      { id: 1, name: 'Laptop', category: 'electronics', price: 1000 },
      { id: 2, name: 'Shirt', category: 'clothing', price: 50 }
    ]);

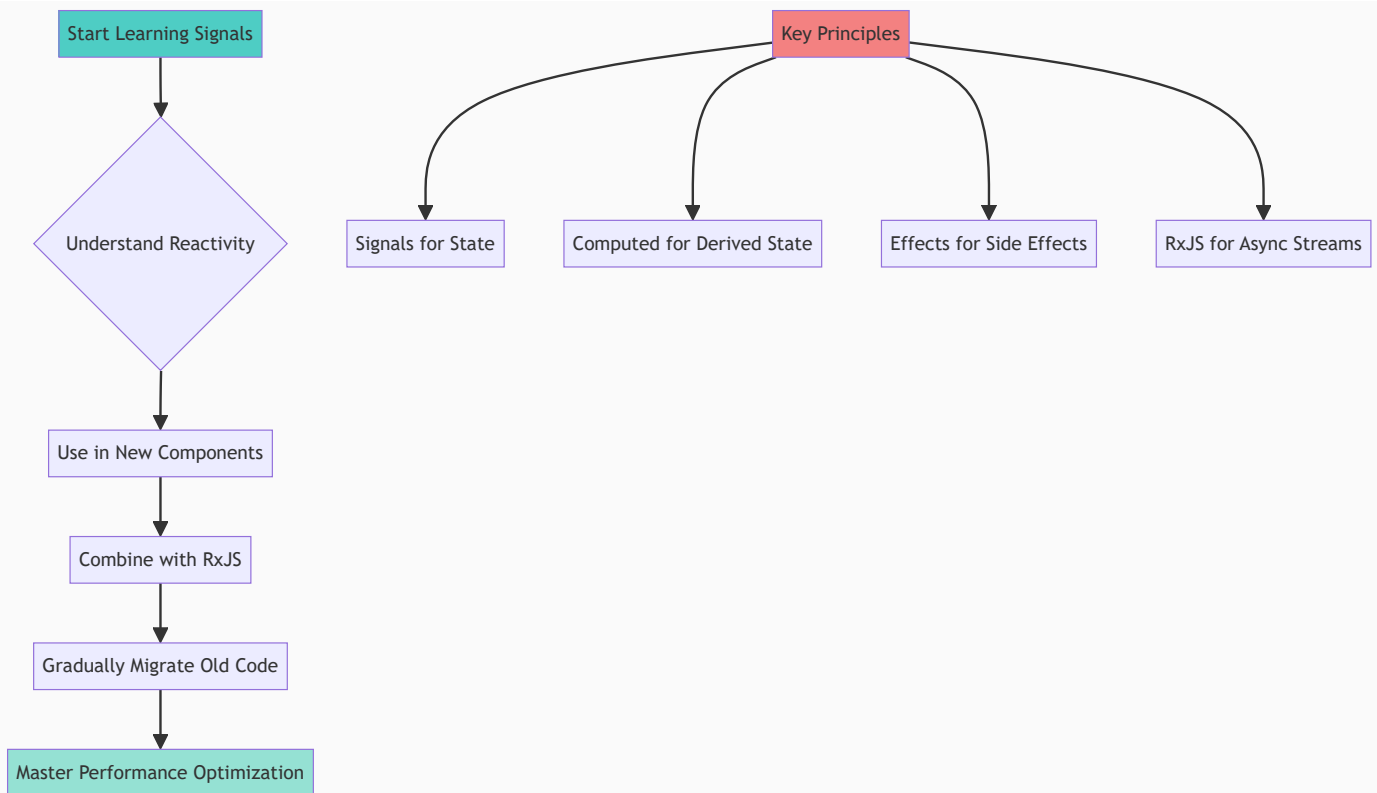
    // Change search query
    component.searchQuery.set('laptop');

    // Computed automatically updates
    expect(component.filteredProducts()).toEqual([
      { id: 1, name: 'Laptop', category: 'electronics', price: 1000 }
    ])
  })
})

```

```
    ]);  
  });  
  
  it('should handle category filtering', () => {  
    const component = new ProductCatalogComponent();  
  
    component.products.set([  
      { id: 1, name: 'Laptop', category: 'electronics', price: 1000 },  
      { id: 2, name: 'Shirt', category: 'clothing', price: 50 }  
    ]);  
  
    component.selectedCategory.set('clothing');  
  
    expect(component.filteredProducts().length).toBe(1);  
    expect(component.filteredProducts()[0].name).toBe('Shirt');  
  });  
});
```

Summary Your Signal Journey



Recap Golden Rules Recap

1. **Signals are for reactive state** - not everything needs to be a signal
2. **Computed is better than effects** - for derived values
3. **RxJS still has its place** - especially for async operations
4. **Immutability matters** - always create new references when updating
5. **OnPush + Signals = Performance** - the winning combination
6. **Backward compatibility is guaranteed** - no need to rush migration
7. **Start small, learn gradually** - don't rewrite everything at once
8. **Type safety is your friend** - leverage TypeScript fully

Next Steps

1. **Experiment:** Create a small component using signals
2. **Compare:** Build the same thing with Zone.js, see the difference
3. **Integrate:** Combine signals with RxJS in a real feature
4. **Optimize:** Use computed and OnPush for performance gains

5. **Share:** Help your team understand these concepts

Resources

- **Official Angular Docs:** <https://angular.dev/guide/signals>
- **Signal RFC:** Understanding the design decisions
- **Community Examples:** GitHub repos showcasing best practices
- **Performance Tools:** Chrome DevTools for measuring improvements

Remember: Signals are a powerful tool, but they're just one tool in your Angular toolbox. The best developers know when to use signals, when to use RxJS, and when to keep things simple. Happy coding! 🚀